

前兩週教案：從線性回歸到 MLP (PyTorch 版)

每週 2 小時 (120 分鐘)，理論與實作交錯，約各半。**兩週主軸**：第一週把「訓練迴圈」這個模板建立起來，第二週只換掉「模型定義」那一行，就從線性回歸長成 MLP。**兩週結束學員能做到**：自己用 `nn.Sequential` 蓋一個 MLP、用標準訓練迴圈訓練它、並套用在回歸與分類兩種任務上。

課前準備 (給家教)

- 環境：建議直接用 Google Colab (免安裝、學員只需瀏覽器)；或本機 `pip install torch matplotlib scikit-learn`
- 先自己把兩週的程式碼跑過一遍，確認版本沒問題
- 準備好可投影 / 共享的 Notebook，邊講邊改

第一週 | 用 PyTorch 學線性回歸 (建立訓練迴圈模板)

本週學習目標

- 理解「模型 → 損失 → 最佳化」這個所有 ML 的共同骨架
- 看懂 PyTorch 的 tensor 與 autograd 在做什麼
- 親手刻出梯度下降，再過渡到 `nn.Linear` 的標準寫法
- 訓練出一個能跑的線性回歸，並讀懂損失曲線

時間總表

時間	項目	形式	分鐘
0:00-0:10	暖身與今日目標	講解	10
0:10-0:40	ML 概念 + 線性回歸三部曲	理論	30
0:40-0:55	PyTorch: tensor 與 autograd 直覺	理論	15
0:55-1:00	休息	—	5
1:00-1:25	實作 ①: 純手動梯度下降	實作	25
1:25-1:35	實作 ②: 改用 autograd	實作	10
1:35-1:55	實作 ③: nn.Linear + optimizer	實作	20
1:55-2:00	收尾、作業、預告	講解	5

分段細節

0:00-0:10 暖身與今日目標 (10 分鐘)

- 一句話定義 ML：不是人寫死規則，而是**從資料把規則學出來**
- 今日目標白話版：「教電腦自己找出一條最能貼合資料的直線」
- 先讓學員看一張「散點 + 一條擬合線」的圖，建立直覺終點

0:10-0:40 ML 概念 + 線性回歸三部曲 (理論, 30 分鐘)

講三件事，這是之後每個模型都會重複的骨架：

1. **模型 (model)**： $\hat{y} = w \cdot x + b$ ，畫出 w 改變斜率、 b 改變高度
2. **損失 (loss)**：均方誤差 $MSE = \text{預測與真實差距的平方平均}$ ；為什麼用平方（懲罰大錯、可微分）
3. **最佳化 (optimization)**：梯度下降。用「在山坡上往最陡的下坡方向走一小步」的比喻；**學習率 = 步伐大小**，太大會跨過頭、太小會走很慢

家教提示：這段是整個課程的地基。多畫圖、少寫公式。梯度下降務必用「下山」比喻講到學員點頭為止。

0:40-0:55 PyTorch: tensor 與 autograd 直覺 (理論, 15 分鐘)

- **tensor**：就像會記錄運算的 NumPy 陣列
- **autograd**：你做的每個運算 PyTorch 都記下來，呼叫 `.backward()` 它就能**自動幫你算出梯度**——這正是上一段「往哪走」的那個方向
- **訓練迴圈骨架**（先口頭講解，等下實作會反覆出現）：

```
forward  → 算預測
loss      → 算差距
backward → 算梯度
step      → 更新參數
```

家教提示：先不要碰程式，純粹把這四步的「為什麼」講清楚。等下三個實作版本都是這四步，只是寫法不同。

0:55-1:00 休息 (5分鐘)

1:00-1:25 實作①：純手動梯度下降 (實作, 25分鐘)

先造一份「我們知道正確答案」的假資料 (真實關係是 $y = 2x + 1$)，這樣最後可以驗證模型有沒有學對。

```
python

import torch
torch.manual_seed(0)

X = torch.linspace(-3, 3, 100).reshape(-1, 1)
y = 2 * X + 1 + 0.5 * torch.randn_like(X) # 真實答案：w=2, b=1，加一點雜訊
```

完全不靠 autograd，連梯度都自己算 (線性回歸的梯度公式很簡單)，讓學員「親手」體會梯度下降：

```
python

w = torch.tensor(0.0)
b = torch.tensor(0.0)
lr = 0.05
losses = []

for epoch in range(200):
    y_pred = w * X + b # forward
    loss = ((y_pred - y) ** 2).mean() # loss (MSE)
    grad_w = (2 * (y_pred - y) * X).mean() # 手算梯度
    grad_b = (2 * (y_pred - y)).mean()
    w -= lr * grad_w # step (手動更新)
    b -= lr * grad_b
    losses.append(loss.item())

print(w.item(), b.item()) # 應該接近 2 和 1
```

接著畫損失曲線，看它一路下降：

```
python

import matplotlib.pyplot as plt
plt.plot(losses); plt.xlabel("epoch"); plt.ylabel("loss"); plt.show()
```

家教提示：這段是本週最有價值的環節，不要省。看到損失曲線往下掉， w 慢慢逼近 2，學員會第一次「感覺到」模型在學習。可以當場把 lr 改成 0.5 (發散) 和 0.001 (龜速)，讓

他們親眼看到學習率的影響。

1:25-1:35 實作②：改用 autograd (實作, 10 分鐘)

只把「手算梯度」換成 `loss.backward()`，其餘邏輯一模一樣——讓學員體會 PyTorch 幫你做了什麼：

```
python

w = torch.tensor(0.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)
lr = 0.05

for epoch in range(200):
    y_pred = w * X + b
    loss = ((y_pred - y) ** 2).mean()
    loss.backward()                # 自動算梯度，不用自己推公式
    with torch.no_grad():
        w -= lr * w.grad
        b -= lr * b.grad
        w.grad.zero_()            # 梯度要手動歸零，否則會累加
        b.grad.zero_()
```

家教提示：強調對照——結果跟手動版一樣，但我們**不用自己推導梯度公式**了。順便解釋 `requires_grad`（要追蹤的參數）和「梯度歸零」這個常見坑。

1:35-1:55 實作③：nn.Linear + optimizer (實作, 20 分鐘)

最後升級到 PyTorch 的慣用寫法。這個訓練迴圈就是接下來每堂課的模板，包括下週的 MLP：

```
python
```

```

import torch.nn as nn

model = nn.Linear(1, 1) # 模型
loss_fn = nn.MSELoss() # 損失
optimizer = torch.optim.SGD(model.parameters(), lr=0.05) # 最佳化器

for epoch in range(200):
    y_pred = model(X) # forward
    loss = loss_fn(y_pred, y) # loss
    optimizer.zero_grad() # 梯度歸零
    loss.backward() # backward
    optimizer.step() # step (更新參數)

print(model.weight.item(), model.bias.item()) # 一樣接近 2 和 1

```

把模型畫在資料上，收個漂亮的尾：

```

python

plt.scatter(X, y, s=8, alpha=0.5)
plt.plot(X, model(X).detach(), color="red")
plt.show()

```

家教提示：明確點出「`nn.Linear` + `optimizer` 幫你包掉了實作①②裡的所有細節」。請把這個四步迴圈框起來、標注「這是模板」，反覆強調下週只會換掉 `model = ...` 那一行。

1:55-2:00 收尾、作業、預告 (5分鐘)

- 回顧：三個版本做的是同一件事，差別只在「自己做多少 vs PyTorch 幫你做多少」
- 預告下週：「如果資料不是一條直線，而是一條彎曲的曲線，這條直線模型會怎樣？」——埋下 MLP 的伏筆

本週作業

- 把學習率調成 0.5、0.01、0.001，各畫一張損失曲線，寫下觀察
- 把資料改成有兩個特徵 (`X` 改成 `100×2`)，用 `nn.Linear(2, 1)` 重跑一次
- (思考題，不用寫程式) 如果真實關係是 $y = x^2$ ，你覺得直線模型擬合得了嗎？為什麼？

第二週 | 從線性回歸長成 MLP

本週學習目標

- 1. 親眼看到線性模型擬合不了非線性資料（建立 MLP 的「動機」）
- 2. 理解 MLP 的核心：隱藏層 + 非線性激活（ReLU），以及「拿掉激活就塌回線性」
- 3. 用 `nn.Sequential` 蓋出第一個 MLP，套用上週的訓練迴圈
- 4. 把 MLP 同時用在回歸和分類兩種任務上

時間總表

時間	項目	形式	分鐘
0:00-0:10	回顧上週 + 今日目標	講解	10
0:10-0:30	線性模型的極限（現場 demo）	理論+實作	20
0:30-0:55	MLP 核心：隱藏層 + 激活函數	理論	25
0:55-1:00	休息	—	5
1:00-1:30	實作 ①：用 MLP 擬合曲線（回歸）	實作	30
1:30-1:55	實作 ②：用 MLP 做分類	實作	25
1:55-2:00	收尾、作業、預告	講解	5

分段細節

0:00-0:10 回顧上週 + 今日目標（10 分鐘）

- 複習那個「模板」訓練迴圈（forward → loss → backward → step）
- 收上週作業的思考題：「`y = x²` 直線擬合得了嗎？」→ 答案是不行，今天就來解決
- 今日目標：把那條只會畫直線的模型，升級成能畫任意曲線的 MLP

0:10-0:30 線性模型的極限（現場 demo，20 分鐘）

造一份彎曲的資料（正弦波），用上週的 `nn.Linear` 去擬合，故意失敗給學員看：

```
python
```

```

import torch, torch.nn as nn
import matplotlib.pyplot as plt
torch.manual_seed(0)

X = torch.linspace(-3, 3, 200).reshape(-1, 1)
y = torch.sin(X) + 0.1 * torch.randn_like(X)  # 一條彎曲的關係

linear = nn.Linear(1, 1)
opt = torch.optim.SGD(linear.parameters(), lr=0.05)
loss_fn = nn.MSELoss()
for epoch in range(300):
    loss = loss_fn(linear(X), y)
    opt.zero_grad(); loss.backward(); opt.step()

plt.scatter(X, y, s=8, alpha=0.4)
plt.plot(X, linear(X).detach(), color="red")  # 只能畫一條直線，蓋不住正弦波
plt.show()

```

家教提示：讓學員看著那條直線「無論怎麼訓練都貼不上波浪」。這個「失敗」是今天的關鍵動機——問他們：缺了什麼？引導到「模型表達能力不夠」。

0:30-0:55 MLP 核心：隱藏層 + 激活函數（理論，25 分鐘）

- **加一層隱藏層**：在輸入和輸出之間多一排神經元，模型就能組合出更複雜的形狀
- **非線性激活 (ReLU) 是關鍵**： $\text{ReLU}(x) = \max(0, x)$ ，把直線「折彎」
- **最重要的觀念**：如果把激活函數拿掉，多層線性層在數學上會塌回一條直線（線性的合成還是線性）。所以「激活函數」正是區分 MLP 和線性回歸的那把鑰匙
- 用上週畫過的網路圖複習：線性回歸 = 單一線性神經元；MLP = 多一層帶 ReLU 的隱藏層
- **反向傳播**：其實就是上週的 `.backward()`，PyTorch 一樣自動幫你算多層的梯度——不是新東西

家教提示：在白板上畫「直線 → (ReLU) → 折線 → 疊很多折線 → 逼近任意曲線」這個過程，是讓學員「秒懂」激活函數為什麼重要的最佳方式。

0:55-1:00 休息（5 分鐘）

1:00-1:30 實作①：用 MLP 擬合曲線（實作，30 分鐘）

重點：訓練迴圈跟上週一模一樣，只換掉模型定義那幾行。

python

```

mlp = nn.Sequential(
    nn.Linear(1, 32),    # 輸入 → 隱藏層 (32 個神經元)
    nn.ReLU(),           # ← 就是這行讓它成為 MLP
    nn.Linear(32, 1),    # 隱藏層 → 輸出
)

optimizer = torch.optim.Adam(mlp.parameters(), lr=0.01) # MLP 常用 Adam，收斂較快
loss_fn = nn.MSELoss()

for epoch in range(1000):    # 跟上週同一個模板
    y_pred = mlp(X)
    loss = loss_fn(y_pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

plt.scatter(X, y, s=8, alpha=0.4)
plt.plot(X, mlp(X).detach(), color="red")    # 這次紅線會貼上正弦波！
plt.show()

```

讓學員動手玩：

- 把隱藏層神經元從 32 改成 4，看曲線變粗糙；改成 128，看更貼合
- 把 `nn.ReLU()` 那行刪掉，看模型立刻塌回直線——親手驗證「沒有激活就退化成線性回歸」

家教提示：刪掉 ReLU 那行的 demo 一定要做，這是把上週圖解「眼見為憑」的關鍵時刻。順帶簡單提一下 Adam 跟上週 SGD 的差別（更聰明的步伐調整），但不用深入。

1:30-1:55 實作②：用 MLP 做分類（實作，25 分鐘）

讓學員體會「同一套 MLP，也能做分類」。用一個容易視覺化的彎月形資料：

python


```

from sklearn.datasets import make_moons
X_np, y_np = make_moons(n_samples=300, noise=0.2, random_state=0)
X = torch.tensor(X_np, dtype=torch.float32)
y = torch.tensor(y_np, dtype=torch.long)

clf = nn.Sequential(
    nn.Linear(2, 16),
    nn.ReLU(),
    nn.Linear(16, 2),          # 兩個類別 → 兩個輸出
)
loss_fn = nn.CrossEntropyLoss() # 分類版的損失（對應回歸的 MSE）
optimizer = torch.optim.Adam(clf.parameters(), lr=0.01)

for epoch in range(500):      # 又是同一個模板
    logits = clf(X)
    loss = loss_fn(logits, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

pred = clf(X).argmax(dim=1)
acc = (pred == y).float().mean()
print(f"準確率: {acc:.2%}")

```

如果時間夠，畫出決策邊界（一條漂亮的曲線把兩群分開），呼應「線性做不到、MLP 做得到」。

家教提示：這段重點不是 cross-entropy 的數學細節（一句話帶過：分類版的損失就好），而是讓學員看到「**訓練迴圈我已經會了，換個資料和損失函數，就能做分類**」。決策邊界的視覺化若時間不夠可留作業。

1:55-2:00 收尾、作業、預告（5分鐘）

- 大回顧：兩週下來，學員已經能蓋 MLP、訓練它、用在回歸和分類——而核心就是那個**重複了無數次的訓練迴圈**
- 預告下一階段：模型評估、過擬合、怎麼判斷模型「學得好不好」

本週作業

1. 把 MLP 加深到三層（多一組 `Linear + ReLU`），比較擬合曲線的效果
2. 完成彎月分類的**決策邊界視覺化**（提示：用網格點丟進模型畫等高線）
3. 拿一個你有興趣的小資料集（scikit-learn 內建的也行），自己套用 MLP 模板跑一次

給家教的兩週總結提醒

1. **「訓練迴圈是模板」**這句話要從第一週講到第二週——它是讓學員兩週就能上手 MLP 的關鍵心法。
2. **兩個必做的 demo**：第一週改學習率看損失曲線、第二週刪掉 ReLU 看模型塌回直線。這兩個「眼見為憑」的時刻，抵得過十張投影片。
3. **節奏彈性**：第二週內容偏滿，若學員消化較慢，分類那段（實作②）可以只示範、把練習挪到作業。
4. **數學深淺看人調**：第一週的手算梯度、cross-entropy 的公式，學員數學強就多講，偏應用就點到為止。